
PARAMETER-EFFICIENT PRETRAINING THROUGH LOW-RANK AND HIGH-RANK ADAPTATION TECHNIQUES

Delyan Hristov
Mentor: Radostin Cholakov

ABSTRACT

Training a large language model has become expensive due to the significant hardware requirements involved. This is why various parameter-efficient fine-tuning techniques, such as Low-Rank Adaptation, have been created. However, while these techniques are highly effective for fine-tuning, they impose certain constraints on the model's capacity to learn new information, making them less suitable for pre-training. To address this, in our study, we explore parameter-efficient training approaches and propose a new method that uses two trainable vectors while keeping the original weight matrix frozen. These vectors are merged with the model multiple times throughout the training process, enabling high-rank adaptation. Despite having fewer trainable parameters, our method proved to be efficient for pre-training large language models.

1 Introduction

As language models get more complex, it becomes really important to find efficient and affordable ways to adapt them for specific tasks. To train a model, we need to update its parameters, which require a lot of GPU power, making it less accessible to the average person. To solve this issue various parameter-efficient fine-tuning techniques (PEFT) were developed. Usually, they employ a smaller set of parameters which are optimized during training. As a result, we update fewer parameters compared to regular training, which reduces the memory footprint. These techniques can be categorized into two types: low-rank and high-rank.

1.1 Rank of a matrix

The rank of a matrix represents the number of linearly independent rows or columns in the matrix. If a row or column is dependent, it can be expressed as a combination of other rows or columns and does not give us new information. For that reason, the rank gives important insight into the linear dependencies in a matrix and the knowledge that has been captured.

1.2 Low-rank adaptation

Low-rank adaptation means that the original weights of the model are frozen and we are updating a smaller number of parameters. One of the most popular methods is LoRA [1], which utilizes two smaller low-rank weight matrices that are updated while keeping the original model parameters fixed. That approach significantly reduces memory usage, because we have to maintain fewer optimizer states, gradients, etc, compared to full training. However, even LoRA can face memory challenges when it is used for larger models. To further reduce the number of trainable parameters and address these issues, Vector-based Random Matrix Adaptation VeRA[2] has been introduced. VeRA uses a single pair of frozen random low-rank matrices shared across all model layers and updates small scaling vectors for layer-specific adaptation.

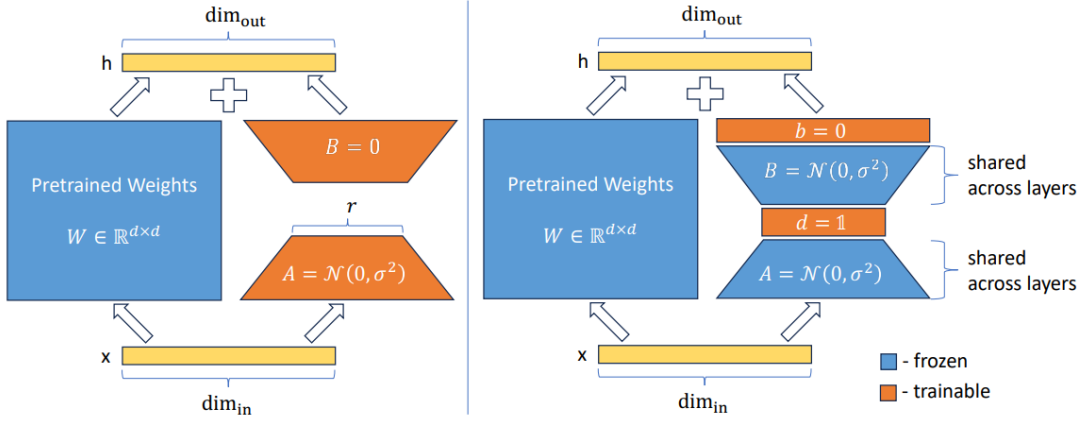


Figure 1: Comparison between LoRA (left) and VeRA (right): LoRA updates the model using two smaller matrices. In contrast, VeRA shares these matrices across all layers and keeps them fixed. During training with VeRA, only two scaling vectors, b and d , are updated.

1.3 High-rank adaptation

While effective for fine-tuning tasks, low-rank updating methods restrict the model’s capacity to learn, which is critical during pre-training phases. This limitation motivates studies to further explore the effectiveness of these methods by proposing efficient ways to perform high-rank updating. That involves adjusting the parameters to capture more complex patterns and details within the data, thereby enhancing the model’s capacity to learn and retain new information. For instance, MoRA [3] demonstrates high-rank adaptation techniques that enable the model to learn new information while being memory efficient. ReLoRA [4] proposes that we can achieve high-rank updates by merging and re-initializing the LoRA matrices. Other techniques[5] explore the effectiveness of random weights and projections for optimization. For example, FLoRA [6] demonstrates how LoRA can be approximated using random projections and performs high-rank adaptation by resampling the projection matrices.

1.4 Problem statement

In this paper, we will further explore the high-rank pre-training and fine-tuning using low-rank matrices. Our objective is to advance current parameter-efficient fine-tuning methods and enhance the performance of large language models by developing a new approach that can be applied during pre-training. This method aims to either increase memory efficiency or improve model accuracy compared to existing techniques, making a parameter-efficient approach that is suitable for pre-training tasks. Inspired by ReLoRA, we propose a similar approach by applying VeRA matrices and merging them into the model multiple times throughout the training process. Our method further reduces the number of trainable parameters compared to ReLoRA. We conducted experiments to test the model’s performance, comparing it to VeRA and other similar techniques.

2 Background

Although large language models (LLMs) and their pre-trained parameters are easily accessible, their significant size makes full fine-tuning for new tasks extremely expensive. This has prompted researchers to explore parameter-efficient fine-tuning and quantization techniques, which aim to reduce the computational requirements and memory overhead.

2.1 LoRA

Low-rank adaptation (LoRA) is usually used for training models on a specific task while reducing the model’s computation demands. That is achieved by freezing the model’s weights and instead optimizing two smaller weight matrices - A and B - for each layer. If the input and the output are d -dimensional vectors of tokens, the first matrix will

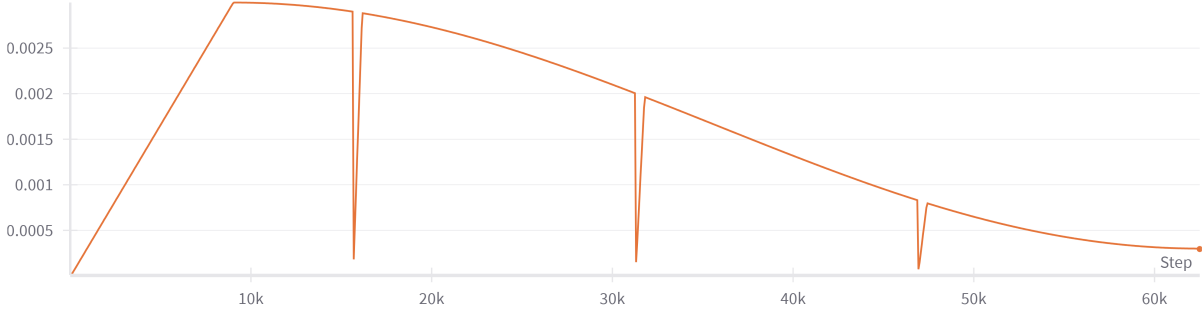


Figure 2: Cosine scheduler used in ReLoRA and our method. Every time, when the additional matrices are merged and reinitialized the learning rate is set to zero followed by a warm-up.

be $A^{d \times r}$, and the second will be $B^{r \times d}$, where r is a hyperparameter that can be changed. LoRA uses random Gaussian initialization for A and zero for B . This allows training to begin with performance equivalent to the original model. During training, the model produces output y based on both the original weights and the new matrices:

$$y = Wx + ABx \quad (1)$$

But then updates only the new matrices. When certain parameters are frozen, computing and storing the optimizer states for those specific parameters is unnecessary, making the process more memory-efficient. The number of trainable parameters is influenced by r and the shape of the original weights. When we multiply A by B we get a new $\Delta W^{d \times d}$ matrix, which has the size of the frozen W matrix. In that way, we are training the model without the necessity of optimizing $d \times d$ parameters. Instead, we can work with $d \times r + r \times d$ parameters. At the end of the training, we can merge the matrices and store only $W + \Delta W$:

$$y = Wx + ABx = (W + AB)x, \quad (2)$$

Which means we are not saving additional parameters. Another advantage is that we can interchange various LoRA matrices for different tasks, which significantly reduces memory usage. By storing only the matrices instead of the entire models, we achieve greater diversity in the final outputs while conserving memory. This approach enables the efficient handling of multiple tasks without the need to store separate models for each task.

2.2 ReLoRA

These matrices can also be employed for high-rank updating, which significantly enhances the model’s ability to learn and adapt to complex tasks. An example of such a method is ReLoRA, which benefits from the ability of LoRA to merge with the original model, utilizing key properties of matrix rank:

$$\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B)). \quad (3)$$

This shows that the rank of a product cannot exceed the rank of either matrix involved.

$$\text{rank}(A + B) \leq \text{rank}(A) + \text{rank}(B) \quad (4)$$

This indicates that the rank of a sum is at most the sum of the ranks of the individual matrices.

These properties demonstrate that summing two low-rank matrices allows us to achieve a higher rank, enabling the model to capture more information. To benefit from that property, ReLoRA uses low-rank matrices, which are periodically merged into the model after a certain number of steps. New ones are then initialized, allowing for continuous adaptation and high-rank updating. This restarting process can be described as:

$$\Delta W = sA^1B^1 + sA^2B^2 + \dots + sA^NB^N, \quad (5)$$

where $s \in \mathbb{R}$ is a fixed scaling factor usually equal to $\frac{1}{r}$, N is the number of resets.

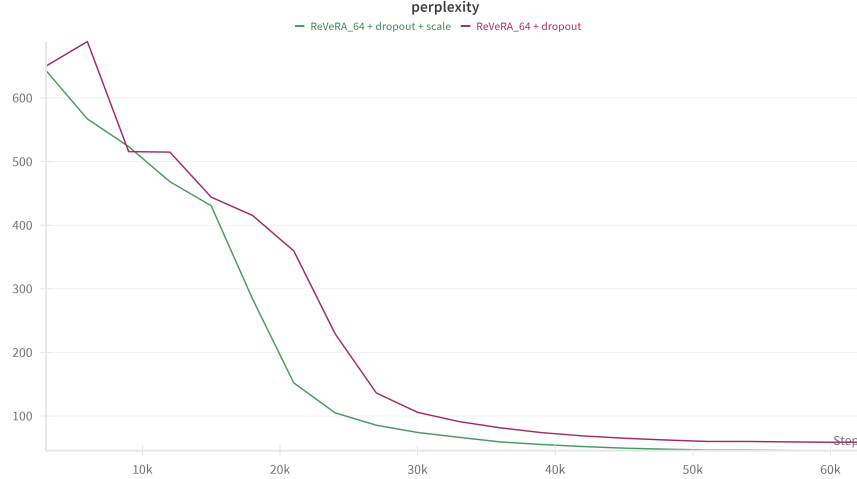


Figure 3: Perplexity comparison between ReVeRA with and without scale on the C4 dataset, illustrating its performance across different training stages.

To update the model parameters we use optimizers like Adam (Adaptive Moment Estimation) [7]. Adam works by adjusting the learning rate for each parameter individually. It calculates two values: the exponentially decaying average of the gradients and the exponentially decaying average of the squared gradients. These two averages, known as the optimizer’s states, help the optimizer decide the best way to update the parameters, making the learning process faster and more efficient. However, even if we keep resetting the LoRA matrices, the optimizer’s stored averages will cause the model to be updated in the same way as with normal LoRA, limiting the effectiveness of the resets. To address these issues, ReLoRA resets a user-configurable percentage of the optimizer states during the merge of the matrices and sets the learning rate to 0, followed by warm-up steps (Figure:2). In that way, they create high-rank adaptation using low-rank matrices. This pretraining approach shows results similar to full-rank training and significantly reduces the computation cost.

2.3 VeRA

Another very useful adapter inspired by LoRA is VeRA. It sets the values for A and B using Kaiming initialization [8]. The values for these matrices are frozen and shared across all layers, which means they remain constant during training and are applied universally throughout the model (Figure: 1). This approach significantly reduces the number of trainable parameters, leading to more efficient computation and memory usage. Then VeRA adds two small trainable vectors d and b , which leads to reducing the number of trainable parameters. The process can be expressed as follows:

$$y = Wx + \Lambda_b A \Lambda_d Bx, \quad (6)$$

Where the vectors Λ_b and Λ_d scale the outputs of these matrices, enabling training without the need to adjust the entire weight matrix. The multiplication is possible because, although Λ_b and Λ_d are originally vectors, they are treated as diagonal matrices during the multiplication. This allows them to scale intermediate vectors without changing their structure. This approach not only reduces the number of trainable parameters but also cuts down on memory usage, as the frozen matrices do not require storing gradients or optimizer states. The outcomes of this process resemble those of training with LoRA but it exhibits a significant reduction in memory usage.

3 Method

In our study, we aimed to create and test a new high-rank adaptation approach. We used BERT-medium, a language model part of the BERT family [9]. Bert-medium is a transformer-based decoder-only language model.

We trained it using Colossal Clean Crawled Corpus (C4)[10]. The dataset consists of over 156 billion tokens gathered from over 365 million domains across the web. It has been used to train models like T5 [11]. The dataset is unlabeled, so we used masked language modelling to train the model. This approach hides a portion of the tokens in each sequence, the task is to predict the masked tokens.

Our method is similar to ReLoRA. We implemented VeRA in all of the attention layers. The difference between VeRA and our approach (ReVeRA) is that, in our method, the new matrices are merged and reinitialized after a certain number of steps. Similarly to ReLoRA, after each merge, we reset a percentage of the optimizer states and set the learning rate to zero, followed by a warm-up phase. This strategy helps to achieve high-rank adaptation with low-rank matrices. Additionally, we introduced another hyperparameter called *ReVeRA_alpha*. This hyperparameter adjusts a value that scales the output, enhancing performance (Figure: 3). Performing this process multiple times brings the total neural network update to:

$$\Delta W = \alpha \Lambda_b^1 A \Lambda_d^1 B + \alpha \Lambda_b^2 A \Lambda_d^2 B + \dots + \alpha \Lambda_b^N A \Lambda_d^N B, \quad (7)$$

Where α is a constant equal to $\frac{\text{ReVeRA_alpha}}{r}$, A and B are the frozen low-rank matrices shared across all layers, and Λ_b and Λ_d are the scaling vectors which are being treated as diagonal matrices during the multiplication. The matrices A and B are initialized using Kaiming initialization. The scaling vector Λ_b^i is set to zero and Λ_d^i to 10^{-1} . By doing so, when we start training, the model will initially perform as the original because the output of the matrices is zero, ensuring that the model’s performance does not degrade at the beginning of the training process. Furthermore, we only need to train the scaling vectors, which makes this approach more memory-efficient than LoRA and ReLoRA (Table: 1).

	ReLoRA		ReVeRA
	r	# Trainable Parameters	# Trainable Parameters
MEDIUM	16	393.2K	12.6K
	64	1572.8K	13.8K
	256	6291.4K	18.4K
BASE	16	884.7K	28.2K
	64	3538.9K	29.9K
	256	14155.7K	36.8K
GPT-3	16	67.8M	3.53M
	64	452.9M	3.55M
	256	1811.9M	3.61M

Table 1: Comparison between trainable parameters for ReLoRA and ReVeRA across different model sizes. We calculate the parameters as if we add the adapters to the query, key, and value layers. The models are BERT-medium, BERT-base and GPT-3 [12]. The table demonstrates that as the value for r increases and model size increases, VeRA becomes more computationally efficient.

We load and train the model using bfloat16 precision. Bfloat16 (brain floating point) [13] is a 16-bit floating point representation that is widely used in deep learning to reduce memory usage and accelerate training. It maintains the same dynamic range as the 32-bit floating point format (float32) but with less precision, often sufficient for training deep learning models. Using bfloat16 helps us save memory and speed up the training process without significantly impacting model performance. For each training technique, we used 62,500 *training steps* with a *batch size* of 64.

4 Results

We trained the model on the C4 dataset and evaluated it using perplexity. Perplexity is defined as the exponentiated average negative log-likelihood of a sequence. It provides insight into how effectively a model predicts or generates sequences. Lower perplexity values indicate that the model performs better. We compared our method to VeRA and ReLoRA. Through this comparison, we aimed to measure the performance of our approach in different configurations and settings, assessing its effectiveness relative to these established methods. For all our experiments, we used brainfloat16. Through experimentation, we identified the most optimal hyperparameters (Table: 3), which led to the best results. Compared to VeRA (Figure: 4) our approach managed to achieve better results while maintaining a similar training time. This indicates that by resetting and merging the matrices, we are effectively obtaining a high-rank update.

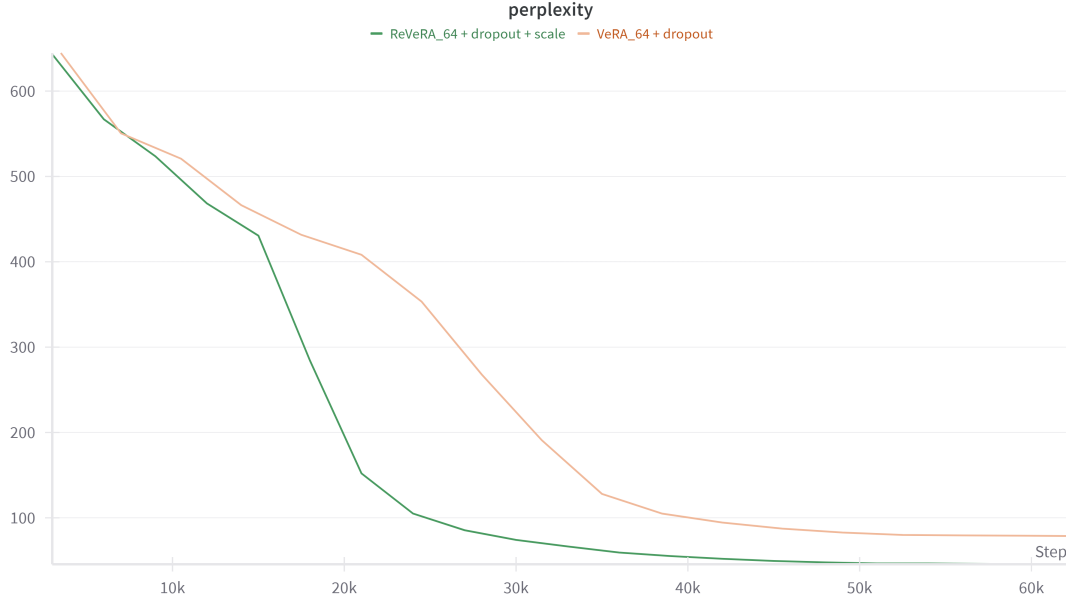


Figure 4: Comparing our method (ReVeRA) with regular VeRA using $r = 64$ and bfloat16 precision.

However, compared to ReLoRA (Table: 2), our method performs worse but uses fewer parameters, making it more suitable for training larger models where it’s preferable to trade off some performance for memory.

Method	Parameters	Perplexity
LoRA	1572.8K	30.40
ReLoRA	1572.8K	18.41
VeRA	13.8K	78.59
ReVeRA	13.8K	45.93

Table 2: Comparing different pretraining approaches on the C4 dataset using $r = 64$ and bfloat16 precision. The parameters column shows the number of trainable parameters each adapter is using. ReVeRA performs better than VeRA but worse than LoRA and ReLoRA. However, it uses fewer parameters, making it a more suitable choice for larger models.

5 Conclusion

This study introduces a new high-rank pre-training method by merging VeRA matrices into the model. Our approach outperforms VeRA, demonstrating that by merging the matrices, we achieve high-rank updating. Moreover, compared to ReLoRA, our technique performs slightly worse but utilizes fewer trainable parameters, making it suitable for situations where computational power is limited. Similarly to ReLoRA, our approach demonstrates that this merging and re-initialization process is ineffective for fine-tuning (Appendix: B).

Acknowledgments

First I want to thank my mentor Radostin Cholakov for proposing this topic, as well as giving crucial advice during the development of the project. I also like to thank Nikola Gyulev, Delyan Boychev, and Emiliana Dimitrova for their general advice on the readability of the paper. We acknowledge the America for Bulgaria and Beautiful Science foundations for partially funding the computational resources used. The Google ML Developer Programs team supported this work by providing Google Cloud and Colab credits.

References

- [1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. 2021.
- [2] Dawid J. Kopiczko, Tijmen Blankevoort, and Yuki M. Asano. Vera: Vector-based random matrix adaptation, 2024.
- [3] Ting Jiang, Shaohan Huang, Shengyue Luo, Zihan Zhang, Haizhen Huang, Furu Wei, Weiwei Deng, Feng Sun, Qi Zhang, Deqing Wang, and Fuzhen Zhuang. Mora: High-rank updating for parameter-efficient fine-tuning, 2024.
- [4] Vladislav Lialin, Namrata Shivagunde, Sherin Muckatira, and Anna Rumshisky. Relora: High-rank training through low-rank updates, 2023.
- [5] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning, 2020.
- [6] Yongchang Hao, Yanshuai Cao, and Lili Mou. Flora: Low-rank adapters are secretly gradient compressors, 2024.
- [7] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, 2015.
- [9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [10] Jesse Dodge, Maarten Sap, Ana Marasović, William Agnew, Gabriel Ilharco, Dirk Groeneveld, Margaret Mitchell, and Matt Gardner. Documenting large webtext corpora: A case study on the colossal clean crawled corpus, 2021.
- [11] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2023.
- [12] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.
- [13] Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, Jiyan Yang, Jongsoo Park, Alexander Heinecke, Evangelos Georganas, Sudarshan Srinivasan, Abhisek Kundu, Misha Smelyanskiy, Bharat Kaul, and Pradeep Dubey. A study of bfloat16 for deep learning training, 2019.
- [14] Alex Warstadt, Amanpreet Singh, and Samuel R. Bowman. Neural network acceptability judgments, 2019.
- [15] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding, 2019.

A Hyperparameters

Hyperparameter	Value
learning rate	3e-3
weight decay	2e-2
warmup steps	9000
cycle length	15625
restart warmup steps	500
ReVeRA alpha	256
ReVeRA dropout	0.1

Table 3: We performed multiple runs to determine the most optimal hyperparameters for ReVeRA with $r = 64$ and a *batch size* = 64. The parameters are presented in the table. The *cycle length* refers to the number of steps before each merging and reinitializing of the VeRA matrices, while *restart warmup steps* is the warmup period after every merge.

B ReVeRA for fine-tuning

To test if ReVeRA is suitable for fine-tuning, we used pre-trained T5-base model [11] and The Corpus of Linguistic Acceptability dataset (COLA) [14] from the General Language Understanding Evaluation benchmark (GLUE) [15]. The dataset consists of 10657 sentences from 23 linguistics publications. Each sentence is annotated with a label indicating whether it is grammatically acceptable or not. The task involves training the model to accurately classify these sentences based on their grammatical acceptability. We tested it using the Matthews correlation coefficient.

r	Vera	ReVeRA (ours)
1024	60.89	59.86
512	60.09	59.52
256	59.81	57.62

Table 4: Comparison between our method (ReVeRa) and VeRA using various values for r on the COLA dataset, with results reported using the Matthews correlation coefficient.

Similarly to ReLoRA, our approach performs worse, indicating that the merging and reinitialization process may not be effective for fine-tuning.